



TourGuide

TourGuide

Documentation fonctionnelle et technique

Sommaire

- 1. Présentation du projet
 - 1.1 Objectifs du projet
 - 1.2 Hors du champ d'application
 - 1.3 Mesures du projet
- 2. Spécifications fonctionnelles
- 3. Spécifications techniques
 - 3.1 Schémas de conception technique
 - 3.2 Solutions techniques

1. Présentation du projet

La solution proposée permet aux utilisateurs de voir quelles sont les attractions touristiques à proximité et d'obtenir des réductions sur les séjours à l'hôtel ainsi que sur les billets de différents spectacles. Le public ciblé est un public touristique.

1.1 Objectifs du projet

Les objectifs du projet sont premièrement de déboguer l'application, certains tests unitaires ne passent pas. Également un des objectif est d'améliorer les performances de l'application. L'appel à deux services particuliers est lent et nécessite une optimisation. Le dernier objectif est de créer un pipeline CI de manière à automatiser l'intégration de l'application.

1.2 Hors du champ d'application

Un des objectifs que nous pourrions envisager est de conteneuriser les services de manière à améliorer la scalabilité de l'application, de passer en architecture full micro-service avec une gateway comme porte d'entrée qui redirigerait vers les différents services. Également, nous pourrions ajouter un système d'authentification et une interface utilisateur.

1.3 Mesures du projet

Le succès du projet sera mesuré premièrement par un succès de tous les tests unitaires déjà écrits. Également un document « Performance » sous forme de classeur Excel permettra d'archiver les performances de l'application en terme d'appel aux services gpsUtil et RewardsCentral.

2. Spécifications fonctionnelles

L'application se présente sous forme d'API REST, les différentes fonctionnalités correspondent donc aux différents endpoints proposés par l'application :

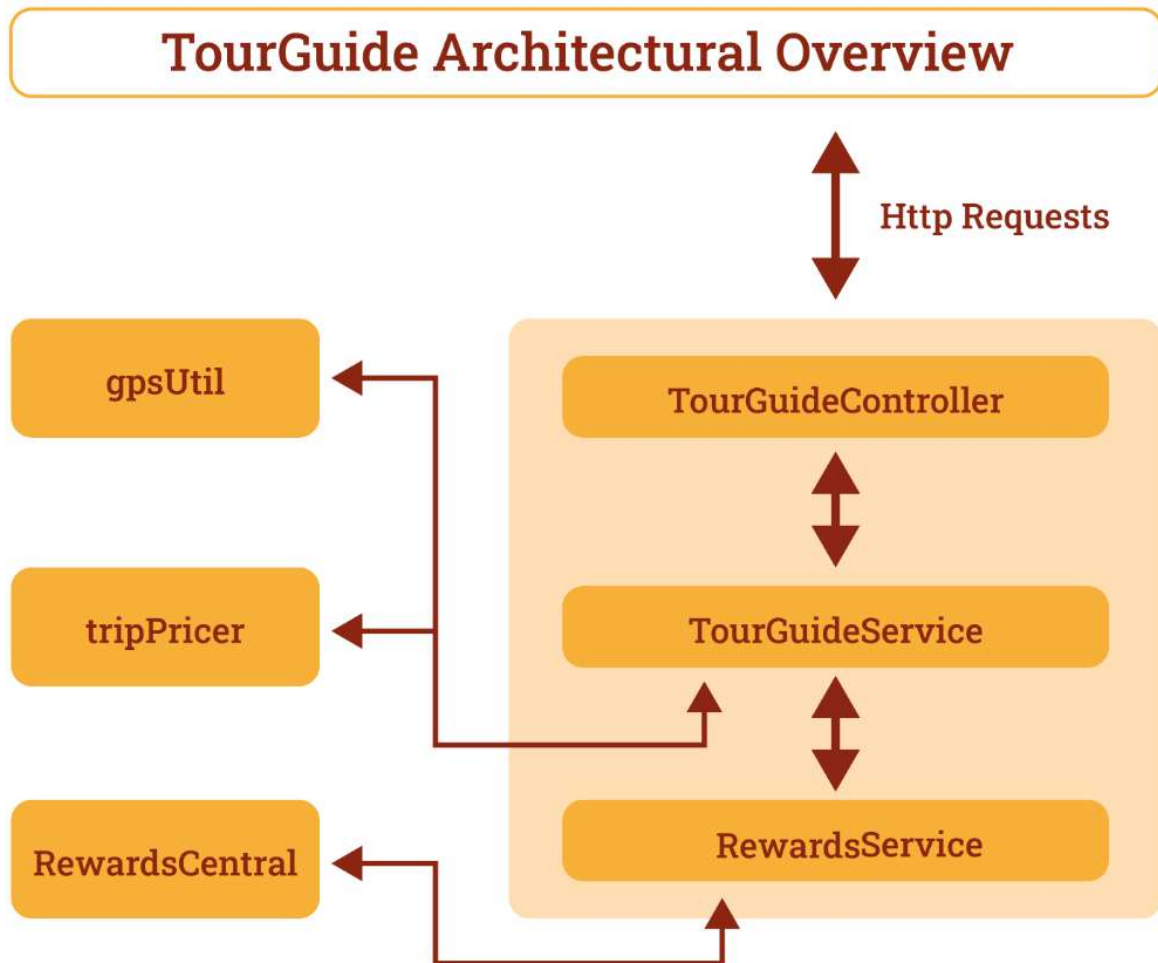
- « /getLocation » : retourne un JSON contenant le dernier lieu visité par l'utilisateur ou le lieu actuel si l'utilisateur n'a pas visité de lieu sous forme de latitude/longitude ainsi qu'une date de visite.
- « /getNearbyAttractions » : retourne une liste des cinq attractions touristiques les plus proches sous forme de JSON contenant :
 - le nom de l'attraction touristique,
 - le lieu de l'attraction touristique (latitude/longitude)
 - le lieu de l'utilisateur (latitude/longitude)
 - la distance en miles entre le lieu de l'utilisateur et le lieu de l'attraction
 - les points de récompense pour l'attraction
- « /getRewards » : retourne une liste de JSON contenant les différents points de récompense

pour chaque attraction visitée par l'utilisateur.

- « /getTripDeals » : retourne une liste de JSON contenant le nom, l'ID et le prix payé pour chaque attraction visitée.

3. Spécifications techniques

3.1 Schémas de conception technique



L'architecture est une application Spring Boot orientée services, avec :

- Un contrôleur REST dans TourGuideController.java
- La logique métier dans TourGuideService.java
- Les calculs des récompenses dans RewardsService.java
- Les données utilisateur en mémoire
- Un accès à des services externes via GpsUtil, RewardCentral et TripPricer

3.2 Solutions techniques

1. Etape 2 : Corrigez le code pour valider les tests

- ▶ `TestRewardsService.nearAllAttractions()` : `ConcurrentModificationException`
 - Modification des getters `User.getVisitedLocations` et `User.getUserRewards` pour renvoyer des copies des listes et pas les listes elles-même.
 - Modification de `User.addUserReward` pour externaliser le test sur l'unicité sinon il y a conflit de concurrence.
 - Modification de `RewardsService.calculateRewards` pour externaliser la logique de test d'unicité et celle d'ajout des rewards (modification de user et utilisation de la variable user)

2. Etape 3 : Implémentez la modification attendue

- ▶ `TourGuideService.getNearByAttractions()` :
 - Tri des attractions par distance par rapport à l'utilisateur.
 - Renvoi des 5 plus proches.
- ▶ Création d'un objet DTO pour satisfaire au TODO : `AttractionInfoResponse`

3. Etape 4 : Améliorez les performances de l'application

- ▶ `TestPerformance.highVolumeTrackLocation()` :
 - Mise en place de la programmation asynchrone avec les `CompletableFuture`.
- ▶ `TestPerformance.highVolumeGetRewards()` :
 - Mise en place de la programmation asynchrone avec les `CompletableFuture`.
 - Fixation du nombre de threads dans le pool.